

Algoritmi e Strutture Dati–parte I

lez. 9

Francesco Camastra

francesco.camastra@uniparthenope.it

Docente

- Prof. Francesco Camastra

Orario lezioni

- Mercoledì 11.00-13.00
(aula 11, Secondo Piano)
- Giovedì 9.00-11.00 (aula 11, Secondo Piano)

Ricevimento

- Orario: Mercoledì 14.00-18.00
- Dove: Stanza 430, Quarto piano, Centro Direzionale, Isola C4
- Per il ricevimento è **OBBLIGATORIO** inviare una email di prenotazione almeno due giorni prima del giorno di ricevimento.
- email del Docente:
francesco.camastra@uniparthenope.it
- Pagina del Docente:
dist.uniparthenope.it/francesco.camastra

Prerequisiti

- Programmazione I
- Matematica I
- Programmazione II
- **NOTA BENE:** Le propedeuticità non sono obbligatorie ... Ossia potreste anche sostenere l'esame senza aver dato Programmazione I e Programmazione II

Frequenza

- La Frequenza del corso non è obbligatoria

Modalità dell' esame

- Prova scritta di Algoritmi e Strutture Dati I ovvero Due prove intercorso (validità quattro appelli)
- Svolgimento di un progetto Algoritmi e Strutture Dati II da scegliersi tra una lista di progetti che verrà comunicata dal docente (validità quattro appelli).
- Superato lo scritto di ASD-I ed il progetto di ASD-II si accede all' Esame Orale congiunto di ASD-I e ASD-II

Testi Consigliati

- ALGORITMI:
- Cormen, Leiserson, Rivest, Stein
*Introduzione agli Algoritmi e
alle strutture Dati*
(seconda edizione o terza edizione)
Ed. Mc Graw-Hill.

Complessità di Max-Heapify

- È descritta dalla ricorrenza:
$$T[n] \leq T(2n/3) + \Theta(1)$$
- La complessità usando il master teorema delle ricorrenze (caso 2) è: $T[n] = \Theta(\log n)$

Complessità Computazionale

- Ricorrenza **Heapify**

$$T(n) = T\left(\frac{2}{3}n\right) + \Theta(1)$$

- Master Teorema delle Ricorrenze

if $T(n) = aT(n/b) + \Theta(n^p)$

dove $p = \log_b a$ then $T(n) = \Theta(\log n \, n^p)$

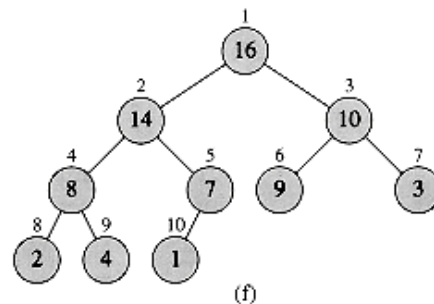
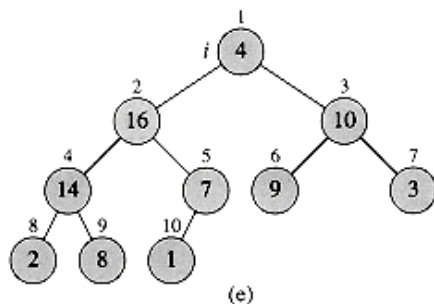
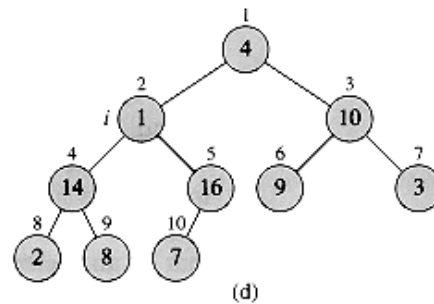
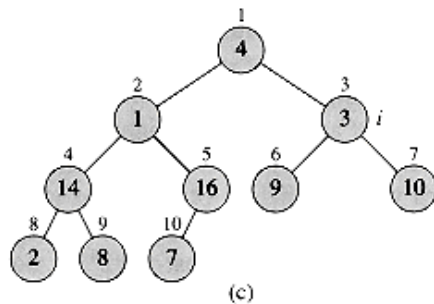
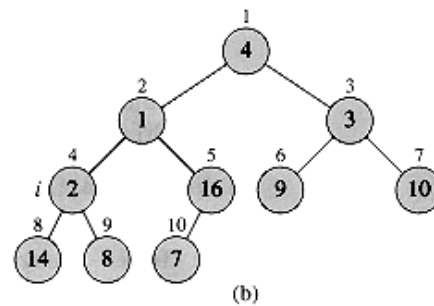
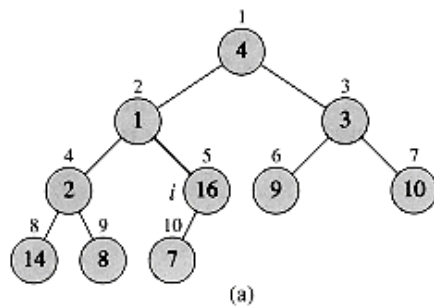
- Poiché $a = 1$ e $b = 3/2$ allora $p = 0$
- La complessità computazionale di **Heapify** è $\Theta(\log n)$.

Build-Max-Heap(*A*)

1. $A.heapsize \leftarrow A.length$
2. **for** $i \leftarrow \left\lfloor \frac{A.length}{2} \right\rfloor$ **downto** 1
3. max-Heapify(A, i)

A

4	1	3	2	16	9	10	14	8	7
---	---	---	---	----	---	----	----	---	---



Build Max Heap: Invariante di Ciclo

- All' inizio di ogni iterazione del ciclo for, righe 2-3, ogni nodo $i + 1, i + 2, \dots, n$ è la radice di un max-heap.

Invariante di Ciclo: Inizializzazione

- Prima dell' iterazione del ciclo, $i = \left\lfloor \frac{n}{2} \right\rfloor$. Ogni nodo $\left\lfloor \frac{n}{2} \right\rfloor + 1, \left\lfloor \frac{n}{2} \right\rfloor + 2, \dots, n$ è una foglia e, quindi, è la radice di un max-heap banale.

Invariante di Ciclo: Conservazione

- Per verificare che ogni iterazione conserva l'invariante di un ciclo, notiamo che i figli del nodo i hanno una numerazione più alta di i . Per l'invariante di ciclo, quindi, essi sono entrambi radici di max-heap. È la condizione richiesta affinché la chiamata MAX-HEAPIFY renda il nodo i la radice di un max-heap.
- .

(cont.)

- Inoltre, la chiamata MAX-HEAPIFY preserva la proprietà che tutti i nodi $i + 1, i + 2, \dots, n$ siano radici di max-heap.
- La diminuzione di i nell'aggiornamento del ciclo for ristabilisce l'invariante di ciclo per la successiva iterazione.

Invariante di Ciclo: Conclusione

- Alla fine del ciclo, $i = 0$. Per l'invariante di ciclo, ogni nodo $1, 2, \dots, n$ è la radice di un max-heap; in particolare, lo è il nodo 1.

Complessità di Build-Max-Heap

- La complessità è $O(n)$.

Complessità di Build-Max-Heap

- Ogni chiamata di MAX-HEAPIFY costa un tempo $O(\log n)$ e ci sono $O(n)$ chiamate.
- Quindi, il tempo di esecuzione è $O(n \log n)$.
- Questo limite superiore, sebbene corretto, non è asintoticamente stretto.

Proprietà di un Heap

- Un heap di n elementi:
 - ha un' **altezza $\lfloor \log n \rfloor$**
 - Per ogni h , **al massimo $\left\lceil \frac{n}{2^{h+1}} \right\rceil$ nodi di altezza h .**

Complessità di Build-Max-Heap

- Il tempo richiesto dalla procedura MAX-HEAPIFY quando viene chiamata per un nodo di altezza h è $O(h)$, quindi il costo totale di BUILD-MAX-HEAP è limitato superiormente da:

$$\sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O \left(\sum_{h=0}^{\lfloor \log n \rfloor} n \left\lceil \frac{h}{2^h} \right\rceil \right)$$

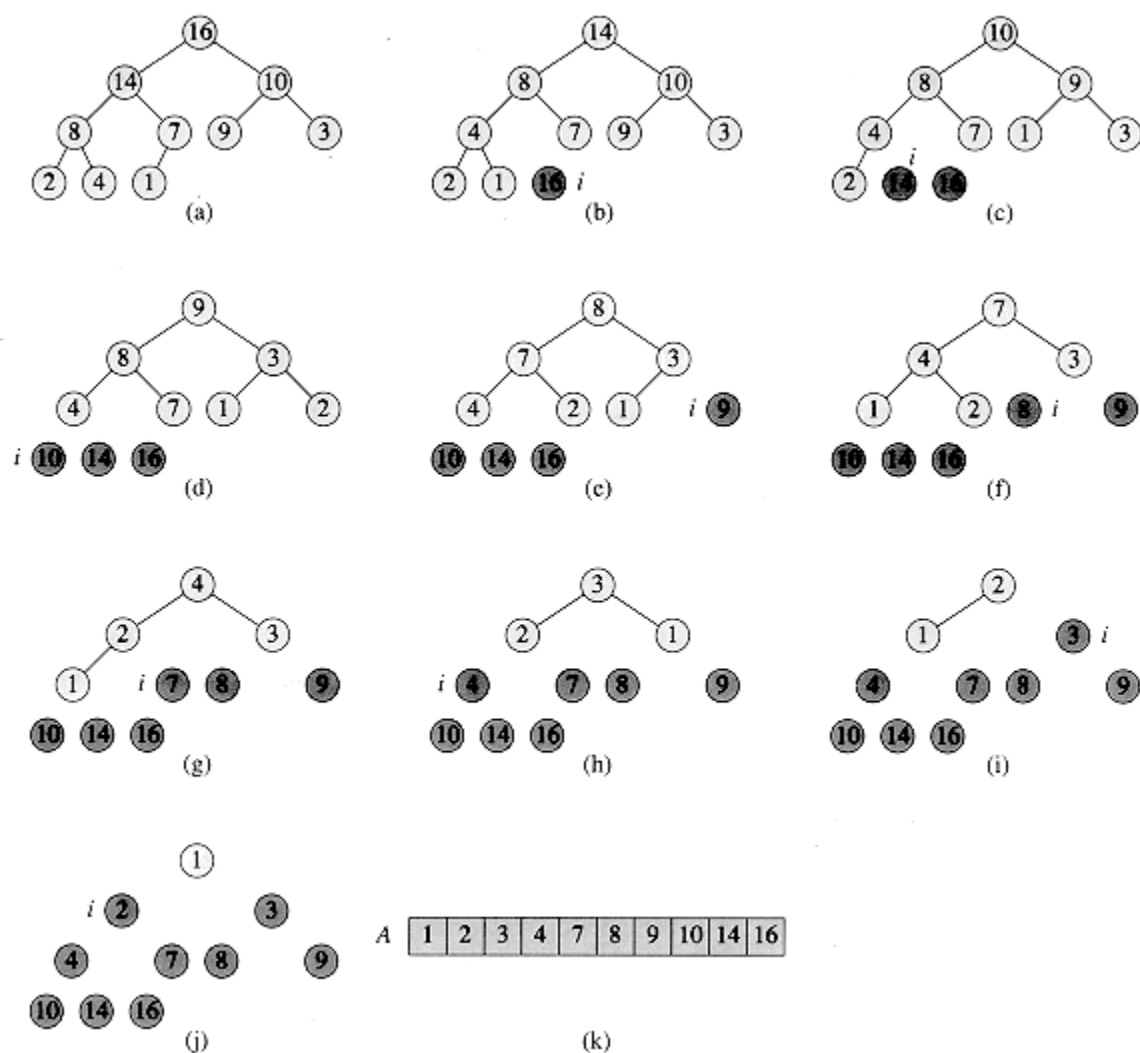
$$\begin{aligned} \sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) &= O \left(n \sum_{h=0}^{\lfloor \infty \rfloor} \frac{h}{2^h} \right) = O \left(n \frac{\frac{1}{2}}{\left(1 - \frac{1}{2}\right)^2} \right) \\ &= O(n) \end{aligned}$$

Serie Geometriche

- Serie armoniche $\sum_{k=1}^n \frac{1}{k} = \log_e n + O(1)$
- Serie geometriche $\sum_{k=0}^n x^k = \frac{x^{n+1} - 1}{x - 1}$
- Serie geometriche infinite $\sum_{k=1}^{\infty} x^k = \frac{1}{1-x}$
- Serie geometriche $\sum_{k=1}^n kx^k = \frac{x}{(1-x)^2}$

Heap-Sort(A)

1. Build-Max-Heap(A)
2. **For** $j \leftarrow A.length$ **downto** 2
3. Scambia $A[1] \leftrightarrow A[j]$
4. $A.heapSize \leftarrow A.heapsize - 1$
5. Max-Heapify($A, 1$)



Complessità di Heap-Sort

- La complessità è $O(n \log n)$.
- La procedura HEAPSORT impiega un tempo $O(n \log n)$, in quanto la chiamata di BUILD_MAX_HEAP impiega $O(n)$ e ciascuna delle $(n - 1)$ chiamate di MAX_HEAPIFY impiega un tempo $O(\log n)$.

Coda con priorità

- È un insieme S di dati x cui è associato un valore detto priorità $key[x]$
- Coda di max-priorità (min-priorità simmetrico):
 - **Insert(S, x)** inserisce l'elemento x nella coda S .
 - **Maximum(S)** restituisce l'elemento con chiave massima.
 - **Extract-Max(S)** rimuove e restituisce l'elemento in S con chiave massima.

(cont.)

- **Increase-priority(S, x, k)** aumenta il valore della chiave di x al nuovo valore k (che si suppone essere grande almeno quanto il valore attuale della chiave di x).

CODA DI MIN PRIORITY

- Una coda di Min priority supporta le seguenti operazioni analoghe:
 - INSERT
 - MINIMUM
 - EXTRACT_MINIMUM
 - DECREASE_KEY

Heap-Maximum(A)

- **return** $A[1]$

Heap-Extract-max(A)

1. **if** $A.heapsize < 1$
2. **error** underflow dell' heap
3. $max \leftarrow A[1]$
4. $A[1] \leftarrow A[heapsize]$
5. $A.heapsize \leftarrow A.heapsize - 1$
6. Max-Heapify($A, 1$)
7. **return** max

Complessità di Heap-Extract-Max

- La complessità di Heap-Extract-Max è $O(\log n)$, in quanto svolge una quantità di costante di lavoro oltre a Max-Heapify.

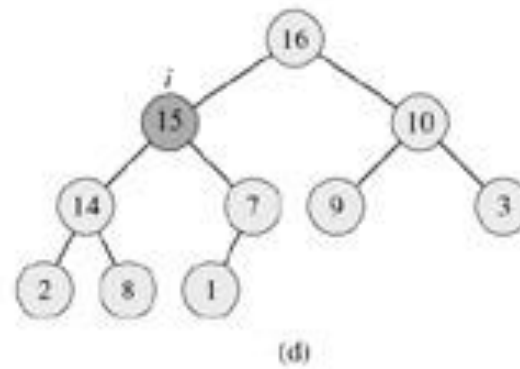
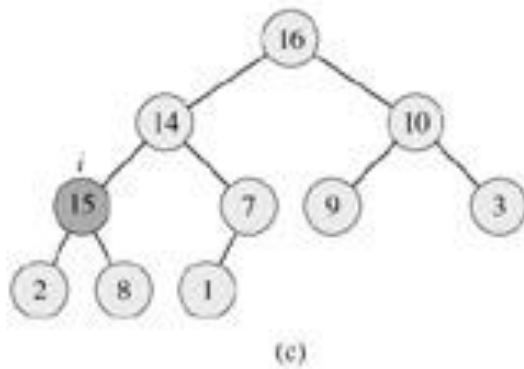
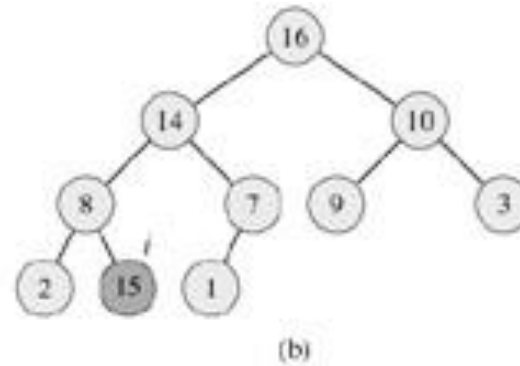
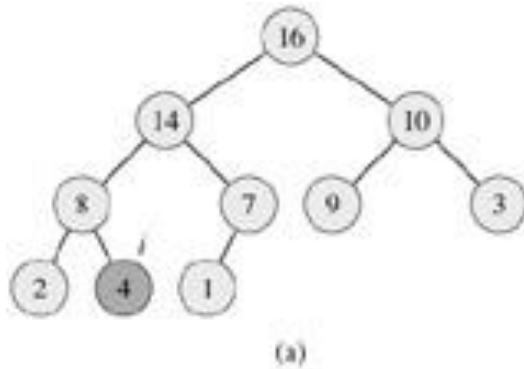
Heap-Increase-Key(A, i, key)

1. **If** $\text{key} < A[i]$
2. **error** “invalid data”
3. $A[i] \leftarrow \text{key}$
4. **while** $i > 1$ **and** $A[\text{parent}(i)] < A[i]$
5. scambia $A[\text{parent}(i)] \leftrightarrow A[i]$
6. $i \leftarrow \text{parent}(i)$

Complessità di HEAP-INCREASE-KEY

- La complessità di HEAP-INCREASE-KEY è $O(\log n)$, in quanto il cammino verso la radice dal nodo aggiornato ha lunghezza $O(\log n)$.

HEAP-INCREASE-KEY



MAX-HEAP-INSERT

- La procedura MAX-HEAP-INSERT implementa l'operazione INSERT.
- Prende come input la chiave del nuovo elemento da inserire nel max-heap A .
- La procedura prima espande il max-heap aggiungendo all'albero una nuova foglia la cui chiave è $-\infty$; poi chiama HEAP-INCREASE-KEY per impostare la chiave di questo nodo e mantenere la proprietà del max-heap.

MAX-HEAP-INSERT(A, key)

1. $A.heapsize \leftarrow A.heapsize + 1$
2. $A[A.heapsize] \leftarrow -\infty$
3. HEAP-INCREASE-KEY($A, A.heapsize, key$)

Complessità di MAX-HEAP-INSERT

- Il tempo di esecuzione della procedura MAX-HEAP-INSERT con un heap di n elementi è $O(\log n)$.

Ordinamento per confronti:

Limite inferiore

- Gli ordinamenti per confronti possono essere visti astrattamente in termini di **alberi di decisione**.
- Quando raggiunge una foglia, l' albero ha trovato il suo ordinamento.

(cont.)

- Poiché qualsiasi algoritmo di ordinamento corretto deve essere in grado di produrre ogni permutazione del suo input, **condizione necessaria è che ciascuna delle $n!$ permutazioni compaia come una foglia dell' albero.**
- E che ciascuna di queste foglie sia raggiungibile attraverso un percorso di nodi che parta dalla radice.

(cont.)

- La lunghezza del cammino semplice più lungo a una delle sue foglie raggiungibili rappresenta il numero dei confronti che svolgerà il corrispondente algoritmo nel caso peggiore.
- Quindi **il numero dei confronti nel caso peggiore è dato dall' altezza del suo albero di decisione.**

Teorema

- **Qualsiasi algoritmo di ordinamento per confronti richiede $\Omega(n \log n)$.**

Dimostrazione

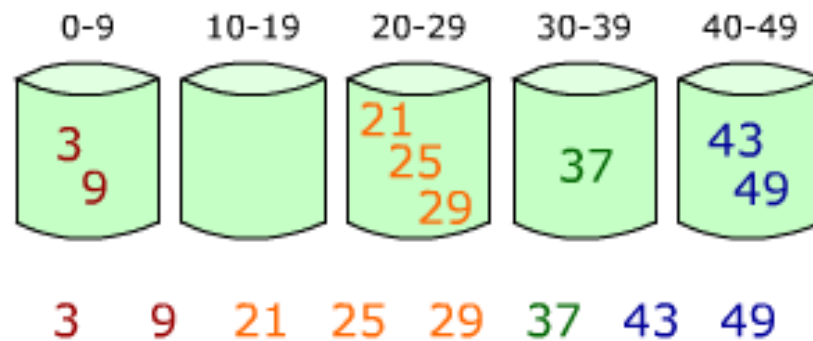
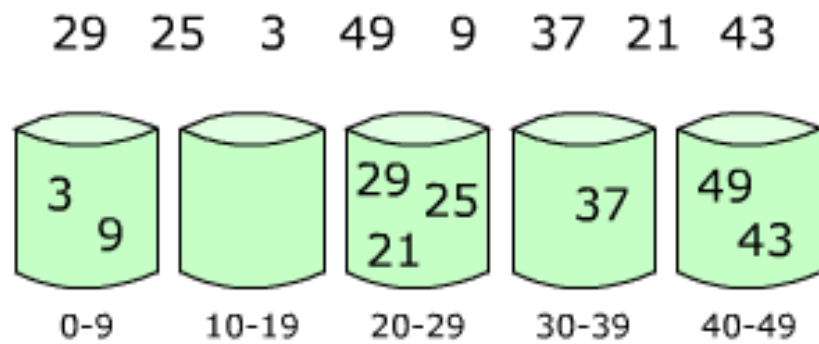
- L' altezza dell' albero di decisione dove ogni permutazione appaia come una foglia raggiungibile fornisce il limite inferiore.
- Supponiamo che l' albero abbia altezza h con l foglie raggiungibili che corrisponde ad un ordinamento di n elementi.
- Ogni permutazione dell' input compare in una foglia, si ha $n! \leq l$. Ma ogni albero ha al più 2^h foglie. Quindi si ha: $n! \leq l \leq 2^h$.

(cont.)

- Ogni permutazione dell' input compare in una foglia, si ha $n! \leq l$. Ma ogni albero ha al più 2^h foglie. Quindi si ha: $n! \leq l \leq 2^h$.
- Ma si ottiene $\log(n!) \leq h$ e ricordando la formula di Stirling ($n! \sim n^n e^{-n}$) abbiamo che $h = \Omega(n \log n)$.

Bucket sort

- **Bucket sort** o **bin sort** è un algoritmo di ordinamento il cui funzionamento consiste nel suddividere un array in un numero finito di buckets. Ciascun bucket è ordinato individualmente o utilizzando un algoritmo di ordinamento differente od applicando ricorsivamente il bucket sort.
- La sua **complessità Media** è $\Theta(n)$.
- Il funzionamento del Bucket sort è il seguente:
 - Definisce un array di "buckets" inizialmente vuoti.
 - Mette ogni elemento dell' array nel suo bucket.
 - Ordina ogni bucket non-vuoto.
 - Sposta gli elementi dai buckets all' array originale



BUCKET_SORT (A)

1. sia $B[0..n - 1]$ un nuovo array
2. $n \leftarrow A.length$
3. for $i \leftarrow 0$ **to** $n - 1$
4. crea la lista vuota $B[i]$
5. for $i \leftarrow 1$ **to** n
6. inserisci $A[i]$ nella lista $B[i]$
7. **for** $j \leftarrow 0$ to $n - 1$
8. Ordina la lista $B[i]$ con insertion-sort
9. Concatena le liste $B[0], B[1], \dots B[n - 1]$ in ordine.

Complessità

- La complessità dell' Insertion sort nel caso peggiore è $O(n^2)$, $\Theta(n)$ quella per suddividere nei bucket, quindi la complessità totale è: $T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$

Valor medio Bucket sort

$$E[T(n)] = E\left[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)\right]$$

$$E[T(n)] = \Theta(n) + E\left[\sum_{i=0}^{n-1} O(n_i^2)\right]$$

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} E[O(n_i^2)]$$

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O(E[n_i^2])$$

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O\left(2 - \frac{1}{n}\right) = \Theta(n)$$

Perché vale $E[n_i^2] = 2 - \frac{1}{n}$ per $0 < i < n-1$

Nota

- Anche se sostituiamo l' insertion sort con il merge sort otteniamo $\Theta(n)$ perché vale $\sum_{i=0}^{n-1} n_i \log n = O(n)$